

Feature extraction

ml_dataset/scripts/feature_extraction_code.py

```
1 #!/usr/bin/env python3
2 """
3 FEATURE EXTRACTION CODE FOR THE PIPELINE
4
5 Output:
6 - participant_features_v6.csv (430 rows x ~130 cols)
7 - movie_features_v6.csv      (10 rows x ~260 cols, mean + std)
8
9 """
10 from __future__ import annotations
11 import argparse
12 import csv
13 import json
14 import math
15 import os
16 import statistics
17 import sys
18 from pathlib import Path
19 from typing import Dict, List, Optional, Tuple
20
21 SCRIPT_DIR = Path(__file__).resolve().parent
22 ML_DIR = SCRIPT_DIR.parent
23 PROJECT = ML_DIR.parent
24 OPENFACE_DIR = PROJECT / "OpenFace"
25 QUANTUM_DIR = PROJECT / "Quantum"
26 STUDY_DIR = PROJECT / "Study_data"
27 META_DIR = ML_DIR / "metadata"
28 OUT_DIR = ML_DIR / "data" / "model_ready" / "movie_success_v6"
29
30 CONDITIONS = [
31     "AMUSEMENT", "ANGER", "AWE", "DISGUST", "ENTHUSIASM",
32     "FEAR", "LIKING", "NEUTRAL", "SADNESS", "SURPRISE",
33 ]
34 # UTILITIES
35
36 def safe_mean(vals):
37     return statistics.mean(vals) if vals else None
38
39 def safe_std(vals):
40     return statistics.stdev(vals) if len(vals) > 1 else None
41
42 def slope(vals):
43     n = len(vals)
44     if n < 2: return None
45     x_mean = (n - 1) / 2.0
46     y_mean = sum(vals) / n
47     num = sum((i - x_mean) * (v - y_mean) for i, v in enumerate(vals))
48     den = sum((i - x_mean) ** 2 for i in range(n))
49     return num / den if den > 0 else 0.0
50
51 def count_peaks(vals, threshold=None):
52     if len(vals) < 3: return 0
53     if threshold is None:
54         threshold = safe_mean(vals) + (safe_std(vals) or 0)
55     return sum(1 for i in range(1, len(vals) - 1)
56               if vals[i] > vals[i-1] and vals[i] > vals[i+1] and vals[i] > threshold)
57
58 def first_above(vals, threshold, sample_rate):
59     for i, v in enumerate(vals):
60         if v > threshold:
61             return i / sample_rate
62     return None
63
64 def fraction_above(vals, threshold):
65     return sum(1 for v in vals if v > threshold) / len(vals) if vals else 0.0
66
67 def fraction_below(vals, threshold):
68     return sum(1 for v in vals if v < threshold) / len(vals) if vals else 0.0
69
70 def half_split_means(vals):
71     if len(vals) < 4: return None, None
72     mid = len(vals) // 2
```

```

73     return safe_mean(vals[:mid]), safe_mean(vals[mid:])
74
75 def rmssd(intervals):
76     if len(intervals) < 2: return None
77     diffs = [(intervals[i+1] - intervals[i]) ** 2 for i in range(len(intervals) - 1)]
78     return math.sqrt(sum(diffs) / len(diffs))
79
80 def pnn50(intervals):
81     if len(intervals) < 2: return None
82     diffs = [abs(intervals[i+1] - intervals[i]) for i in range(len(intervals) - 1)]
83     return sum(1 for d in diffs if d > 0.050) / len(diffs)
84
85 def dominant_runs(labels):
86     if not labels: return 0, None
87     runs = 1
88     lengths = [1]
89     for i in range(1, len(labels)):
90         if labels[i] != labels[i-1]:
91             runs += 1
92             lengths.append(1)
93         else:
94             lengths[-1] += 1
95     return runs, safe_mean(lengths)
96
97 def delta(stim_val, base_val):
98     """Return stim - base if both numeric, else None."""
99     if stim_val is None or base_val is None:
100         return None
101     try:
102         return float(stim_val) - float(base_val)
103     except (ValueError, TypeError):
104         return None
105
106 # OPENFACE - shared core extractor for stimulus + baseline
107
108 def _openface_core(fpath: Path, duration_s_fallback: float = 60.0) -> Dict:
109     """Returns raw AU/pose/gaze series from an OpenFace CSV."""
110     if not fpath.exists():
111         return {}
112     rows = []
113     with open(fpath) as f:
114         reader = csv.DictReader(f)
115         for row in reader:
116             rows.append(row)
117     if len(rows) < 30:
118         return {}
119
120     fps = 60.0
121     n = len(rows)
122     duration_s = n / fps
123
124     AU_INTENSITY = ["AU01_r", "AU02_r", "AU04_r", "AU05_r", "AU06_r", "AU07_r",
125                     "AU09_r", "AU10_r", "AU12_r", "AU14_r", "AU15_r", "AU17_r",
126                     "AU20_r", "AU23_r", "AU25_r", "AU26_r", "AU45_r"]
127
128     au_series = {}
129     for au in AU_INTENSITY:
130         col = " " + au
131         vals = []
132         for r in rows:
133             try:
134                 vals.append(float(r.get(col, r.get(au, 0))))
135             except (ValueError, TypeError):
136                 vals.append(0.0)
137         au_series[au] = vals
138
139     pose = {"pose_Rx": [], "pose_Ry": [], "pose_Rz": []}
140     for r in rows:
141         for pc in pose:
142             col = " " + pc
143             try:
144                 pose[pc].append(float(r.get(col, r.get(pc, 0))))
145             except (ValueError, TypeError):
146                 pose[pc].append(0.0)
147
148     gaze_x, gaze_y, conf = [], [], []
149     for r in rows:
150         try:

```

```

151         gaze_x.append(float(r.get(" gaze_angle_x", r.get("gaze_angle_x", 0))))
152         gaze_y.append(float(r.get(" gaze_angle_y", r.get("gaze_angle_y", 0))))
153         conf.append(float(r.get(" confidence", r.get("confidence", 0))))
154     except (ValueError, TypeError):
155         gaze_x.append(0.0); gaze_y.append(0.0); conf.append(0.0)
156
157     return {
158         "n": n, "fps": fps, "duration_s": duration_s,
159         "au": au_series, "pose": pose,
160         "gaze_x": gaze_x, "gaze_y": gaze_y, "conf": conf,
161     }
162
163
164 def _openface_level_features(core: Dict) -> Dict[str, Optional[float]]:
165     """Compute just the LEVEL features (used for baseline comparison)."""
166     if not core:
167         return {}
168     au = core["au"]
169     feat = {}
170     # Smile
171     feat["smile_fraction"] = fraction_above(au["AU12_r"], 0.5)
172     duchenne = [1 if au["AU12_r"][i] > 0.5 and au["AU06_r"][i] > 0 else 0
173                 for i in range(core["n"])]
174     feat["duchenne_fraction"] = sum(duchenne) / core["n"]
175     # AU means (for bc versions)
176     for key in ["AU04_r", "AU09_r", "AU12_r", "AU15_r", "AU25_r"]:
177         feat[f"{key.lower()}_mean"] = safe_mean(au[key])
178     # Blink rate
179     au45_active = [1 if v > 0.5 else 0 for v in au["AU45_r"]]
180     blink_onsets = sum(1 for i in range(1, core["n"])
181                       if au45_active[i] == 1 and au45_active[i-1] == 0)
182     feat["blink_rate_per_min"] = blink_onsets / core["duration_s"] * 60
183     return feat
184
185
186 def extract_openface_stim(pid: int, cond: str,
187                          baseline_levels: Dict) -> Dict[str, Optional[float]]:
188     """Stimulus-stage OpenFace features + baseline-corrected versions."""
189     fpath = OPENFACE_DIR / str(pid) / f"{pid}_{cond}_STIMULUS.csv"
190     core = _openface_core(fpath)
191     if not core:
192         return {}
193
194     feat = {}
195     prefix = "of_"
196     au = core["au"]
197     n = core["n"]
198     fps = core["fps"]
199     duration_s = core["duration_s"]
200
201     # Smile dynamics
202     au12, au06 = au["AU12_r"], au["AU06_r"]
203     smile_threshold = 0.5
204     smile_active = [1 if v > smile_threshold else 0 for v in au12]
205     smile_onsets = sum(1 for i in range(1, n)
206                      if smile_active[i] == 1 and smile_active[i-1] == 0)
207     feat[prefix + "smile_rate_per_min"] = smile_onsets / duration_s * 60
208
209     smile_fraction = fraction_above(au12, smile_threshold)
210     feat[prefix + "smile_fraction"] = smile_fraction
211     feat[prefix + "smile_fraction_bc"] = delta(smile_fraction, baseline_levels.get("smile_fraction"))
212
213     # Duchenne smile: AU12 > 0.5 AND AU06 > 0
214     duchenne = [1 if au12[i] > smile_threshold and au06[i] > 0 else 0 for i in range(n)]
215     duchenne_frac = sum(duchenne) / n
216     feat[prefix + "duchenne_fraction"] = duchenne_frac
217     feat[prefix + "duchenne_fraction_bc"] = delta(duchenne_frac, baseline_levels.get("duchenne_fraction"))
218
219     # Episode durations
220     episode_lengths = []
221     current_len = 0
222     for v in smile_active:
223         if v == 1:
224             current_len += 1
225         elif current_len > 0:
226             episode_lengths.append(current_len / fps); current_len = 0
227     if current_len > 0:
228         episode_lengths.append(current_len / fps)

```

```

229 feat[prefix + "smile_mean_duration_s"] = safe_mean(episode_lengths)
230 feat[prefix + "smile_longest_s"] = max(episode_lengths) if episode_lengths else 0.0
231 feat[prefix + "smile_onset_latency_s"] = first_above(au12, smile_threshold, fps)
232 feat[prefix + "smile_peak_timing"] = (au12.index(max(au12)) / n) if au12 else None
233
234 first_half, second_half = half_split_means(au12)
235 feat[prefix + "smile_first_half"] = first_half
236 feat[prefix + "smile_second_half"] = second_half
237 if first_half is not None and second_half is not None:
238     feat[prefix + "smile_buildup"] = second_half - first_half
239
240 # — AU velocities (dynamics) —
241 for au_name in ["AU12_r", "AU04_r", "AU09_r", "AU25_r"]:
242     series = au[au_name]
243     velocity = [abs(series[i] - series[i-1]) for i in range(1, len(series))]
244     short = au_name.lower().replace("_r", "")
245     feat[prefix + f"{short}_velocity"] = safe_mean(velocity)
246
247 # — Individual negative AU means (DON'T sum them) —
248 # AU04 = brow furrow (concern, concentration)
249 # AU09 = nose wrinkle (disgust)
250 # AU15 = lip corner depress (sadness)
251 for au_name, label in [("AU04_r", "brow_furrow"), ("AU09_r", "nose_wrinkle"),
252                     ("AU15_r", "lip_depress")]:
253     series = au[au_name]
254     m = safe_mean(series)
255     feat[prefix + f"{label}_mean"] = m
256     feat[prefix + f"{label}_max"] = max(series) if series else None
257     feat[prefix + f"{label}_mean_bc"] = delta(m, baseline_levels.get(f"{au_name.lower()}_mean"))
258     feat[prefix + f"{label}_onset_latency_s"] = first_above(series, 1.0, fps)
259
260 # — Blink features —
261 au45 = au["AU45_r"]
262 blink_active = [1 if v > 0.5 else 0 for v in au45]
263 blink_onsets = sum(1 for i in range(1, n)
264                   if blink_active[i] == 1 and blink_active[i-1] == 0)
265 blink_rate = blink_onsets / duration_s * 60
266 feat[prefix + "blink_rate_per_min"] = blink_rate
267 feat[prefix + "blink_rate_bc"] = delta(blink_rate, baseline_levels.get("blink_rate_per_min"))
268
269 blink_times = [i / fps for i in range(1, n)
270               if blink_active[i] == 1 and blink_active[i-1] == 0]
271 ibis = [blink_times[i] - blink_times[i-1] for i in range(1, len(blink_times))]
272 feat[prefix + "blink_interval_std"] = safe_std(ibis)
273
274 # — Head movement —
275 pose = core["pose"]
276 head_vel = []
277 for i in range(1, n):
278     v = sum(abs(pose[pc][i] - pose[pc][i-1]) for pc in pose)
279     head_vel.append(v)
280 feat[prefix + "head_velocity_mean"] = safe_mean(head_vel)
281 # Fixed: use fraction_below directly, not convoluted inverse
282 feat[prefix + "head_stillness_fraction"] = fraction_below(head_vel, 0.01)
283
284 # — Gaze stability (combined horizontal + vertical) —
285 gx, gy = core["gaze_x"], core["gaze_y"]
286 gaze_mag = [math.sqrt(gx[i]**2 + gy[i]**2) for i in range(len(gx))]
287 feat[prefix + "gaze_stability"] = safe_std(gaze_mag)
288
289 # — Confidence (quality indicator) —
290 feat[prefix + "confidence_mean"] = safe_mean(core["conf"])
291
292 return feat
293
294
295 def extract_openface_baseline(pid: int) -> Dict[str, Optional[float]]:
296     """Extract the level features only from the BASELINE OpenFace file."""
297     fpath = OPENFACE_DIR / str(pid) / f"{pid}_BASELINE_STIMULUS.csv"
298     core = _openface_core(fpath)
299     return _openface_level_features(core)
300
301 # QUANTUM — baseline + stimulus
302
303 def _quantum_core(fpath: Path):
304     if not fpath.exists():
305         return {}
306     with open(fpath) as f:

```

```

307     data = json.load(f)
308     frames = data.get("frames", [])
309     if len(frames) < 30:
310         return {}
311
312     EMOTIONS = ["neutral", "anger", "disgust", "happiness", "sadness", "surprise"]
313     n = len(frames)
314     fps = 60.0
315     duration_s = n / fps
316
317     emo_series = {e: [] for e in EMOTIONS}
318     dominant = []
319     yaw, pitch = [], []
320     for frame in frames:
321         faces = frame.get("faces", [])
322         if not faces:
323             for e in EMOTIONS:
324                 emo_series[e].append(0.0)
325                 dominant.append("no_face"); continue
326         face = faces[0]
327         em = face.get("emotions", {})
328         for e in EMOTIONS:
329             emo_series[e].append(float(em.get(e, 0)))
330         dominant.append(face.get("emotionName", "neutral"))
331         hp = face.get("headPose", {})
332         if hp:
333             yaw.append(float(hp.get("yaw", 0)))
334             pitch.append(float(hp.get("pitch", 0)))
335
336     return {
337         "n": n, "fps": fps, "duration_s": duration_s,
338         "emo": emo_series, "dominant": dominant,
339         "yaw": yaw, "pitch": pitch, "emotions": EMOTIONS,
340     }
341
342
343 def _quantum_level_features(core):
344     if not core: return {}
345     return {f"{e}_mean": safe_mean(core["emo"][e]) for e in core["emotions"]}
346
347
348 def extract_quantum_stim(pid, cond, baseline_levels):
349     fpath = QUANTUM_DIR / str(pid) / f"{pid}_{cond}_STIMULUS.json"
350     core = _quantum_core(fpath)
351     if not core: return {}
352
353     feat = {}
354     prefix = "q_"
355     n = core["n"]; fps = core["fps"]; duration_s = core["duration_s"]
356     emo = core["emo"]
357
358     # Per-emotion stats + baseline-corrected
359     for e in core["emotions"]:
360         m = safe_mean(emo[e])
361         feat[prefix + f"{e}_mean"] = m
362         feat[prefix + f"{e}_max"] = max(emo[e]) if emo[e] else None
363         feat[prefix + f"{e}_mean_bc"] = delta(m, baseline_levels.get(f"{e}_mean"))
364
365     # Engagement dynamics
366     non_neutral = [sum(emo[e][i] for e in core["emotions"] if e != "neutral")
367                    for i in range(n)]
368     feat[prefix + "non_neutral_fraction"] = fraction_above(non_neutral, 0.1)
369     feat[prefix + "neutral_escape_time_s"] = first_above(non_neutral, 0.2, fps)
370
371     # Entropy (with proper normalisation)
372     entropies = []
373     for i in range(n):
374         probs = [emo[e][i] for e in core["emotions"]]
375         total = sum(probs)
376         if total > 0:
377             probs = [p / total for p in probs]
378             h = -sum(p * math.log2(p + 1e-10) for p in probs if p > 0)
379             entropies.append(h)
380     feat[prefix + "entropy_mean"] = safe_mean(entropies)
381
382     # Transitions – use RATE per minute, not raw count
383     transitions, mean_run_len = dominant_runs(core["dominant"])
384     feat[prefix + "transition_rate_per_min"] = transitions / duration_s * 60 if duration_s > 0 else None

```

```

385     feat[prefix + "mean_dominant_duration_s"] = (mean_run_len / fps) if mean_run_len else None
386
387     # Happiness dynamics
388     h = emo["happiness"]
389     feat[prefix + "happiness_onset_s"] = first_above(h, 0.1, fps)
390     feat[prefix + "happiness_slope"] = slope(h)
391     h_first, h_second = half_split_means(h)
392     feat[prefix + "happiness_buildup"] = (h_second - h_first) if h_first is not None and h_second is not
None else None
393
394     # Peak timing
395     total_emo = [sum(emo[e][i] for e in core["emotions"] if e != "neutral") for i in range(n)]
396     if total_emo:
397         feat[prefix + "peak_emotion_timing"] = total_emo.index(max(total_emo)) / n
398
399     # Head pose stability
400     feat[prefix + "head_yaw_std"] = safe_std(core["yaw"])
401     feat[prefix + "head_pitch_std"] = safe_std(core["pitch"])
402
403     return feat
404
405
406 def extract_quantum_baseline(pid):
407     fpath = QUANTUM_DIR / str(pid) / f"{pid}_BASELINE_STIMULUS.json"
408     core = _quantum_core(fpath)
409     return _quantum_level_features(core)
410 # EMPATICA - baseline + stimulus
411
412 def _empatika_core(fpath):
413     if not fpath.exists():
414         return {}
415     with open(fpath) as f:
416         data = json.load(f)
417
418     eda_vals = [float(x[1]) for x in data.get("EDA", []) if len(x) >= 2]
419     ibi_vals = [float(x[1]) for x in data.get("IBI", [])
420                 if len(x) >= 2 and 0.3 < float(x[1]) < 2.0]
421     temp_vals = [float(x[1]) for x in data.get("TEMP", []) if len(x) >= 2]
422     bvp_vals = [float(x[1]) for x in data.get("BVP", []) if len(x) >= 2]
423     acc_raw = data.get("ACC", [])
424     acc_mag = [math.sqrt(float(x[1])**2 + float(x[2])**2 + float(x[3])**2)
425               for x in acc_raw if len(x) >= 4]
426
427     return {"eda": eda_vals, "ibi": ibi_vals, "temp": temp_vals,
428            "bvp": bvp_vals, "acc_mag": acc_mag}
429
430
431 def _empatika_level_features(core):
432     if not core: return {}
433     feat = {}
434     feat["eda_mean"] = safe_mean(core["eda"]) if len(core["eda"]) > 10 else None
435
436     # SCR rate per minute
437     if len(core["eda"]) > 10:
438         scr_count = count_peaks(core["eda"])
439         feat["scr_rate_per_min"] = scr_count / (len(core["eda"]) / 4.0) * 60
440     else:
441         feat["scr_rate_per_min"] = None
442
443     if len(core["ibi"]) > 5:
444         feat["hr_mean"] = 60.0 / safe_mean(core["ibi"])
445         feat["rmssd"] = rmssd(core["ibi"])
446         feat["pnn50"] = pnn50(core["ibi"])
447     else:
448         feat["hr_mean"] = feat["rmssd"] = feat["pnn50"] = None
449
450     feat["temp_mean"] = safe_mean(core["temp"]) if len(core["temp"]) > 10 else None
451     feat["movement_mean"] = safe_mean(core["acc_mag"]) if len(core["acc_mag"]) > 10 else None
452     return feat
453
454
455 def extract_empatika_stim(pid, cond, baseline_levels):
456     fpath = STUDY_DIR / str(pid) / f"{pid}_{cond}_STIMULUS_EMPATICA.json"
457     core = _empatika_core(fpath)
458     if not core: return {}
459     feat = {}
460     prefix = "emp_"
461     eda = core["eda"]; ibi = core["ibi"]; temp = core["temp"]

```

```

462     bvp = core["bvp"]; acc_mag = core["acc_mag"]
463
464     # EDA
465     if len(eda) > 10:
466         eda_mean = safe_mean(eda)
467         feat[prefix + "eda_mean"] = eda_mean
468         feat[prefix + "eda_mean_bc"] = delta(eda_mean, baseline_levels.get("eda_mean"))
469         feat[prefix + "eda_std"] = safe_std(eda)
470         feat[prefix + "eda_slope"] = slope(eda)
471
472         scr_count = count_peaks(eda)
473         scr_rate = scr_count / (len(eda) / 4.0) * 60
474         feat[prefix + "scr_rate_per_min"] = scr_rate
475         feat[prefix + "scr_rate_bc"] = delta(scr_rate, baseline_levels.get("scr_rate_per_min"))
476
477         threshold = safe_mean(eda) + (safe_std(eda) or 0)
478         feat[prefix + "scr_onset_latency_s"] = first_above(eda, threshold, 4.0)
479
480         first, second = half_split_means(eda)
481         feat[prefix + "eda_buildup"] = (second - first) if first is not None and second is not None else
None
482
483     # HRV
484     if len(ibi) > 5:
485         hr = 60.0 / safe_mean(ibi)
486         feat[prefix + "hr_mean"] = hr
487         feat[prefix + "hr_mean_bc"] = delta(hr, baseline_levels.get("hr_mean"))
488         feat[prefix + "ibi_mean"] = safe_mean(ibi)
489         feat[prefix + "ibi_std"] = safe_std(ibi)
490
491         r = rmssd(ibi)
492         feat[prefix + "rmssd"] = r
493         feat[prefix + "rmssd_bc"] = delta(r, baseline_levels.get("rmssd"))
494
495         p = pnn50(ibi)
496         feat[prefix + "pnn50"] = p
497         feat[prefix + "pnn50_bc"] = delta(p, baseline_levels.get("pnn50"))
498
499         hr_vals = [60.0 / v for v in ibi if v > 0]
500         hr_first, hr_second = half_split_means(hr_vals)
501         feat[prefix + "hr_reactivity"] = (hr_second - hr_first) if hr_first is not None and hr_second is
not None else None
502         feat[prefix + "hr_range"] = max(hr_vals) - min(hr_vals) if hr_vals else None
503
504     # BVP
505     feat[prefix + "bvp_std"] = safe_std(bvp) if len(bvp) > 10 else None
506
507     # Temperature – absolute level, baseline-corrected (NOT slope)
508     if len(temp) > 10:
509         t = safe_mean(temp)
510         feat[prefix + "temp_mean"] = t
511         feat[prefix + "temp_mean_bc"] = delta(t, baseline_levels.get("temp_mean"))
512
513     # Movement / fidgeting
514     if len(acc_mag) > 10:
515         m = safe_mean(acc_mag)
516         feat[prefix + "movement_mean"] = m
517         feat[prefix + "movement_mean_bc"] = delta(m, baseline_levels.get("movement_mean"))
518         feat[prefix + "movement_std"] = safe_std(acc_mag)
519         # Fidget rate per minute
520         fc = count_peaks(acc_mag)
521         feat[prefix + "fidget_rate_per_min"] = fc / (len(acc_mag) / 32.0) * 60
522
523     return feat
524
525
526 def extract_empatica_baseline(pid):
527     fpath = STUDY_DIR / str(pid) / f"{pid}_BASELINE_STIMULUS_EMPATICA.json"
528     return _empatica_level_features(_empatica_core(fpath))
529
530
531 #
532 # MUSE EEG – baseline + stimulus
533 #
534
535 def _muse_core(fpath):
536     if not fpath.exists(): return {}
537     with open(fpath) as f:

```



```

538     data = json.load(f)
539
540     required = ["Alpha_AF7", "Alpha_AF8", "Beta_AF7", "Beta_AF8",
541               "Theta_AF7", "Theta_AF8",
542               "Alpha_TP9", "Alpha_TP10", "HeadBandOn"]
543     for r in required:
544         if r not in data or not data[r]:
545             return {}
546
547     n = len(data["Alpha_AF7"])
548     if n < 100: return {}
549
550     hb = data.get("HeadBandOn", [1] * n)
551
552     def get_valid(key):
553         vals = data.get(key, [])
554         return [float(vals[i]) for i in range(min(len(vals), n))
555                if i < len(hb) and float(hb[i]) > 0.5]
556
557     return {
558         "alpha_af7": get_valid("Alpha_AF7"), "alpha_af8": get_valid("Alpha_AF8"),
559         "alpha_tp9": get_valid("Alpha_TP9"), "alpha_tp10": get_valid("Alpha_TP10"),
560         "beta_af7": get_valid("Beta_AF7"), "beta_af8": get_valid("Beta_AF8"),
561         "theta_af7": get_valid("Theta_AF7"), "theta_af8": get_valid("Theta_AF8"),
562     }
563
564
565 def _muse_level_features(core):
566     if not core or len(core["alpha_af7"]) < 50: return {}
567     feat = {}
568     # Alpha asymmetry: ln(AF8) - ln(AF7)
569     alpha_asym = []
570     for i in range(min(len(core["alpha_af7"]), len(core["alpha_af8"]))):
571         if core["alpha_af7"][i] > 0 and core["alpha_af8"][i] > 0:
572             alpha_asym.append(math.log(core["alpha_af8"][i]) - math.log(core["alpha_af7"][i]))
573     feat["alpha_asym_mean"] = safe_mean(alpha_asym)
574
575     beta_asym = []
576     for i in range(min(len(core["beta_af7"]), len(core["beta_af8"]))):
577         if core["beta_af7"][i] > 0 and core["beta_af8"][i] > 0:
578             beta_asym.append(math.log(core["beta_af8"][i]) - math.log(core["beta_af7"][i]))
579     feat["beta_asym_mean"] = safe_mean(beta_asym)
580
581     # Frontal alpha (target of suppression)
582     frontal_alpha = [(core["alpha_af7"][i] + core["alpha_af8"][i]) / 2
583                     for i in range(min(len(core["alpha_af7"]), len(core["alpha_af8"])))]
584     feat["frontal_alpha_mean"] = safe_mean(frontal_alpha)
585
586     # Frontal theta
587     frontal_theta = [(core["theta_af7"][i] + core["theta_af8"][i]) / 2
588                     for i in range(min(len(core["theta_af7"]), len(core["theta_af8"])))]
589     feat["frontal_theta_mean"] = safe_mean(frontal_theta)
590
591     # Per-channel alpha means
592     feat["alpha_af7_mean"] = safe_mean(core["alpha_af7"])
593     feat["alpha_af8_mean"] = safe_mean(core["alpha_af8"])
594     feat["alpha_tp9_mean"] = safe_mean(core["alpha_tp9"])
595     feat["alpha_tp10_mean"] = safe_mean(core["alpha_tp10"])
596
597     return feat
598
599
600 def extract_muse_stim(pid, cond, baseline_levels):
601     fpath = STUDY_DIR / str(pid) / f"{pid}_{cond}_STIMULUS_MUSE.json"
602     core = _muse_core(fpath)
603     if not core or len(core["alpha_af7"]) < 50: return {}
604
605     feat = {}
606     prefix = "eeg_"
607
608     # Alpha asymmetry (absolute + BASELINE-CORRECTED)
609     alpha_asym = []
610     for i in range(min(len(core["alpha_af7"]), len(core["alpha_af8"]))):
611         if core["alpha_af7"][i] > 0 and core["alpha_af8"][i] > 0:
612             alpha_asym.append(math.log(core["alpha_af8"][i]) - math.log(core["alpha_af7"][i]))
613
614     asym_mean = safe_mean(alpha_asym)
615     feat[prefix + "alpha_asym_mean"] = asym_mean

```



```

616 feat[prefix + "alpha_asym_change"] = delta(asym_mean, baseline_levels.get("alpha_asym_mean"))
617 feat[prefix + "alpha_asym_std"] = safe_std(alpha_asym)
618 feat[prefix + "alpha_asym_slope"] = slope(alpha_asym) if len(alpha_asym) > 10 else None
619
620 first, second = half_split_means(alpha_asym)
621 feat[prefix + "alpha_asym_buildup"] = (second - first) if first is not None and second is not None
else None
622
623 # Beta asymmetry (+ bc)
624 beta_asym = []
625 for i in range(min(len(core["beta_af7"]), len(core["beta_af8"]))):
626     if core["beta_af7"][i] > 0 and core["beta_af8"][i] > 0:
627         beta_asym.append(math.log(core["beta_af8"][i]) - math.log(core["beta_af7"][i]))
628 beta_mean = safe_mean(beta_asym)
629 feat[prefix + "beta_asym_mean"] = beta_mean
630 feat[prefix + "beta_asym_change"] = delta(beta_mean, baseline_levels.get("beta_asym_mean"))
631
632 # Frontal alpha SUPPRESSION = baseline - stim (positive = more alertness during stim)
633 frontal_alpha_stim = [(core["alpha_af7"][i] + core["alpha_af8"][i]) / 2
634                       for i in range(min(len(core["alpha_af7"]), len(core["alpha_af8"])))]
635 fa_mean = safe_mean(frontal_alpha_stim)
636 feat[prefix + "frontal_alpha_mean"] = fa_mean
637 base_fa = baseline_levels.get("frontal_alpha_mean")
638 if fa_mean is not None and base_fa is not None:
639     feat[prefix + "frontal_alpha_suppression"] = base_fa - fa_mean
640
641 # Frontal theta change (positive = more emotional processing)
642 frontal_theta_stim = [(core["theta_af7"][i] + core["theta_af8"][i]) / 2
643                      for i in range(min(len(core["theta_af7"]), len(core["theta_af8"])))]
644 ft_mean = safe_mean(frontal_theta_stim)
645 feat[prefix + "frontal_theta_mean"] = ft_mean
646 feat[prefix + "frontal_theta_change"] = delta(ft_mean, baseline_levels.get("frontal_theta_mean"))
647
648 # Per-channel alpha (baseline-corrected versions)
649 for ch in ["af7", "af8", "tp9", "tp10"]:
650     m = safe_mean(core[f"alpha_{ch}"])
651     feat[prefix + f"alpha_{ch}_mean"] = m
652     feat[prefix + f"alpha_{ch}_mean_bc"] = delta(m, baseline_levels.get(f"alpha_{ch}_mean"))
653
654 # Alpha variability (time-domain, no baseline needed)
655 feat[prefix + "alpha_variability"] = safe_std(frontal_alpha_stim)
656
657 return feat
658
659
660 def extract_muse_baseline(pid):
661     fpath = STUDY_DIR / str(pid) / f"{pid}_BASELINE_STIMULUS_MUSE.json"
662     return _muse_level_features(_muse_core(fpath))
663
664 # SAMSUNG WATCH - baseline + stimulus
665
666 def _samsung_core(fpath):
667     if not fpath.exists(): return {}
668     with open(fpath) as f:
669         data = json.load(f)
670     hr = [float(x[1]) for x in data.get("heartRate", [])]
671     if len(x) >= 2 and 30 < float(x[1]) < 200:
672         ppi = [float(x[1]) / 1000.0 for x in data.get("PPInterval", [])]
673         if len(x) >= 2 and 300 < float(x[1]) < 2000:
674             acc_raw = data.get("acc", [])
675             acc_mag = [math.sqrt(float(x[1])**2 + float(x[2])**2 + float(x[3])**2)
676                       for x in acc_raw if len(x) >= 4]
677             gyr_raw = data.get("gyr", [])
678             gyr_mag = [math.sqrt(float(x[1])**2 + float(x[2])**2 + float(x[3])**2)
679                       for x in gyr_raw if len(x) >= 4]
680     return {"hr": hr, "ppi": ppi, "acc_mag": acc_mag, "gyr_mag": gyr_mag}
681
682
683 def _samsung_level_features(core):
684     if not core: return {}
685     feat = {}
686     feat["hr_mean"] = safe_mean(core["hr"]) if len(core["hr"]) > 10 else None
687     feat["ppi_mean"] = safe_mean(core["ppi"]) if len(core["ppi"]) > 10 else None
688     feat["movement_mean"] = safe_mean(core["acc_mag"]) if len(core["acc_mag"]) > 30 else None
689     return feat
690
691
692 def extract_samsung_stim(pid, cond, baseline_levels):

```

```

693 fpath = STUDY_DIR / str(pid) / f"{pid}_{cond}_STIMULUS_SAMSUNG_WATCH.json"
694 core = _samsung_core(fpath)
695 if not core: return {}
696
697 feat = {}
698 prefix = "sw_"
699
700 hr = core["hr"]
701 if len(hr) > 10:
702     m = safe_mean(hr)
703     feat[prefix + "hr_mean"] = m
704     feat[prefix + "hr_mean_bc"] = delta(m, baseline_levels.get("hr_mean"))
705     feat[prefix + "hr_std"] = safe_std(hr)
706     feat[prefix + "hr_range"] = max(hr) - min(hr)
707     first, second = half_split_means(hr)
708     feat[prefix + "hr_reactivity"] = (second - first) if first is not None and second is not None else
None
709     feat[prefix + "hr_peak_timing"] = hr.index(max(hr)) / len(hr)
710
711 ppi = core["ppi"]
712 if len(ppi) > 10:
713     m = safe_mean(ppi)
714     feat[prefix + "ppi_mean"] = m
715     feat[prefix + "ppi_mean_bc"] = delta(m, baseline_levels.get("ppi_mean"))
716     feat[prefix + "ppi_std"] = safe_std(ppi)
717     feat[prefix + "ppi_rmssd"] = rmssd(ppi)
718
719 acc_mag = core["acc_mag"]
720 if len(acc_mag) > 30:
721     m = safe_mean(acc_mag)
722     feat[prefix + "movement_mean"] = m
723     feat[prefix + "movement_mean_bc"] = delta(m, baseline_levels.get("movement_mean"))
724     feat[prefix + "movement_std"] = safe_std(acc_mag)
725     # Burst rate per minute
726     bc = count_peaks(acc_mag)
727     feat[prefix + "movement_burst_rate_per_min"] = bc / (len(acc_mag) / 33.4) * 60
728
729 if len(core["gyr_mag"]) > 30:
730     feat[prefix + "gyro_variability"] = safe_std(core["gyr_mag"])
731
732 return feat
733
734
735 def extract_samsung_baseline(pid):
736     fpath = STUDY_DIR / str(pid) / f"{pid}_BASELINE_STIMULUS_SAMSUNG_WATCH.json"
737     return _samsung_level_features(_samsung_core(fpath))
738
739 # QUESTIONNAIRE (demographics, self-report, context)
740
741 def load_questionnaire_data():
742     meta_dir = PROJECT / "ml_dataset" / "metadata"
743     demo = {}
744     with open(meta_dir / "participants.csv") as f:
745         for row in csv.DictReader(f):
746             pid = int(row["participant_id"])
747             demo[pid] = row
748
749 NEVER = 24 * 60 # 1440 min: encode "never today" for missing physiological state
750
751 def parse_mins(s, default_if_missing=None):
752     if not s or not s.strip():
753         return default_if_missing
754     parts = s.strip().split(":")
755     try:
756         return int(parts[0]) * 60 + int(parts[1])
757     except (ValueError, IndexError):
758         return default_if_missing
759
760 quest_data = {}
761 for pid_dir in sorted(STUDY_DIR.iterdir()):
762     if not pid_dir.is_dir() or not pid_dir.name.isdigit():
763         continue
764     pid = int(pid_dir.name)
765     qpath = pid_dir / f"{pid}_QUESTIONNAIRES.json"
766     if not qpath.exists():
767         continue
768     with open(qpath) as f:
769         qdata = json.load(f)

```

```

770     meta = qdata.get("metadata", {})
771     movie_order = meta.get("movie_order", [])
772     familiarity = meta.get("movies_seen_before_study", {})
773     d = demo.get(pid, {})
774
775     baseline_emo = {}
776     for entry in qdata.get("questionnaires", []):
777         if entry["movie"] == "BASELINE":
778             baseline_emo = entry.get("emotions", {})
779             break
780
781     # Parse physiological state ONCE per participant (same for all conditions)
782     # MISSING caffeine/cigarette/activity → 1440 (never today)
783     participant_state = {
784         "participant_age": int(d.get("age", 0)) if d.get("age") else None,
785         "participant_gender": 1 if d.get("gender") == "male" else 0,
786         "wearing_glasses": 1 if meta.get("wearing_glasses") else 0,
787         "minutes_from_wakeup": parse_mins(meta.get("time_from_wake_up", "")),
788         # Missing = "never today" → encode as NEVER (not imputed median)
789         "minutes_from_caffeine": parse_mins(meta.get("time_from_caffeine", "")),
790         "minutes_from_cigarette": parse_mins(meta.get("time_from_cigarette", "")),
791         "minutes_from_activity": parse_mins(meta.get("time_from_activity", "")),
792         "minutes_from_meal": parse_mins(meta.get("time_from_meal", "")),
793     }
794
795     for entry in qdata.get("questionnaires", []):
796         cond = entry["movie"]
797         if cond == "BASELINE":
798             continue
799         emotions = entry.get("emotions", {})
800         sam = entry.get("sam", {})
801         feat = dict(participant_state)
802         # Self-report emotions
803         for emo, val in emotions.items():
804             feat[f"sr_emo_{emo.lower()}"] = val
805             feat[f"sr_emo_bc_{emo.lower()}"] = val - baseline_emo.get(emo, 0)
806         feat["sr_sam_valence"] = sam.get("VALENCE")
807         feat["sr_sam_arousal"] = sam.get("AROUSAL")
808         feat["sr_sam_motivation"] = sam.get("MOTIVATION")
809         feat["viewing_order"] = movie_order.index(cond) if cond in movie_order else None
810         feat["movie_familiarity"] = familiarity.get(cond, 0)
811         quest_data[(pid, cond)] = feat
812
813     return quest_data
814
815 # MAIN
816
817 def main():
818     parser = argparse.ArgumentParser()
819     parser.add_argument("--pid", type=int, help="Single participant ID (debug)")
820     parser.add_argument("--condition", type=str, help="Single condition (debug)")
821     args = parser.parse_args()
822
823     # Condition → movie mapping
824     cond_to_movie = {}
825     with open(META_DIR / "scenes.csv") as f:
826         for row in csv.DictReader(f):
827             if row["scene_id"] != "S000":
828                 cond_to_movie[row["condition"]] = row["movie_id"]
829
830     # Targets
831     targets = {}
832     with open(META_DIR / "movie_financials_placeholder.csv") as f:
833         for row in csv.DictReader(f):
834             if row["movie_id"] and row["movie_id"] != "M000":
835                 targets[row["movie_id"]] = {
836                     "budget_usd": row.get("budget_usd", ""),
837                     "revenue_usd": row.get("revenue_usd", ""),
838                     "roi_percent": row.get("roi_percent", ""),
839                     "success_class": row.get("success_class", ""),
840                 }
841
842     pids = sorted([int(d) for d in os.listdir(str(STUDY_DIR)) if d.isdigit()])
843     if args.pid:
844         pids = [args.pid]

```

```

845 conditions = [args.condition.upper()] if args.condition else CONDITIONS
846
847 print(f"V6 extraction: {len(pids)} participants × {len(conditions)} conditions", flush=True)
848 print("Loading questionnaire data...", flush=True)
849 quest_lookup = load_questionnaire_data()
850
851 all_records = []
852 for pi, pid in enumerate(pids):
853     # Extract baseline features ONCE per participant
854     bl_of = extract_openface_baseline(pid)
855     bl_q = extract_quantum_baseline(pid)
856     bl_emp = extract_empatika_baseline(pid)
857     bl_muse = extract_muse_baseline(pid)
858     bl_sw = extract_samsung_baseline(pid)
859
860     for cond in conditions:
861         mid = cond_to_movie.get(cond, "")
862         if not mid or mid not in targets:
863             continue
864
865         of_feat = extract_openface_stim(pid, cond, bl_of)
866         q_feat = extract_quantum_stim(pid, cond, bl_q)
867         emp_feat = extract_empatika_stim(pid, cond, bl_emp)
868         muse_feat = extract_muse_stim(pid, cond, bl_muse)
869         sw_feat = extract_samsung_stim(pid, cond, bl_sw)
870         quest = quest_lookup.get((pid, cond), {})
871
872         # Require at least ONE modality to exist for this row
873         if not any([of_feat, q_feat, emp_feat, muse_feat, sw_feat]):
874             continue
875
876         record = {
877             "participant_id": pid,
878             "condition": cond,
879             "movie_id": mid,
880             # Modality availability masks
881             "has_openface": 1 if of_feat else 0,
882             "has_quantum": 1 if q_feat else 0,
883             "has_empatika": 1 if emp_feat else 0,
884             "has_muse": 1 if muse_feat else 0,
885             "has_samsung": 1 if sw_feat else 0,
886         }
887         record.update(of_feat)
888         record.update(q_feat)
889         record.update(emp_feat)
890         record.update(muse_feat)
891         record.update(sw_feat)
892         record.update(quest)
893         all_records.append(record)
894
895         if (pi + 1) % 5 == 0 or pi == len(pids) - 1:
896             print(f" [{pi+1}/{len(pids)}] processed · {len(all_records)} records", flush=True)
897
898 if not all_records:
899     print("ERROR: no records", flush=True)
900     sys.exit(1)
901
902 # Collect all feature columns
903 id_cols = {"participant_id", "condition", "movie_id"}
904 feature_cols = sorted(set(k for r in all_records for k in r if k not in id_cols))
905
906 print(f"\nParticipant-level: {len(all_records)} rows × {len(feature_cols)} features", flush=True)
907
908 OUT_DIR.mkdir(parents=True, exist_ok=True)
909 out_cols = ["participant_id", "condition", "movie_id"] + feature_cols
910 p_path = OUT_DIR / "participant_features_v6.csv"
911 with open(p_path, "w", newline="") as f:
912     writer = csv.DictWriter(f, fieldnames=out_cols, extrasaction="ignore")
913     writer.writeheader()
914     for r in all_records:
915         writer.writerow({k: (round(v, 6) if isinstance(v, float) else v)
916                          for k, v in r.items()})
917 print(f"Wrote {p_path}", flush=True)
918
919 # Aggregate to movie level
920 from collections import defaultdict
921 by_movie = defaultdict(list)
922 for r in all_records:

```

```

923     by_movie[r["movie_id"]].append(r)
924
925     numeric_feat_cols = []
926     for col in feature_cols:
927         vals = [r.get(col) for r in all_records if r.get(col) is not None and r.get(col) != ""]
928         if vals:
929             try:
930                 float(vals[0])
931                 numeric_feat_cols.append(col)
932             except (ValueError, TypeError):
933                 pass
934
935     movie_records = []
936     for mid in sorted(targets.keys()):
937         records = by_movie.get(mid, [])
938         if not records:
939             continue
940         row = {"movie_id": mid, "condition": records[0]["condition"],
941               "n_participants": len(records)}
942         row.update(targets[mid])
943         for col in numeric_feat_cols:
944             vals = [float(r[col]) for r in records
945                     if r.get(col) is not None and r[col] != ""]
946             if vals:
947                 row[f"{col}__mean"] = round(statistics.mean(vals), 6)
948                 row[f"{col}__std"] = round(statistics.stdev(vals), 6) if len(vals) > 1 else 0.0
949             else:
950                 row[f"{col}__mean"] = None
951                 row[f"{col}__std"] = None
952         movie_records.append(row)
953
954     if movie_records:
955         m_cols = list(movie_records[0].keys())
956         m_path = OUT_DIR / "movie_features_v6.csv"
957         with open(m_path, "w", newline="") as f:
958             writer = csv.DictWriter(f, fieldnames=m_cols)
959             writer.writeheader()
960             for r in movie_records:
961                 writer.writerow({k: (round(v, 6) if isinstance(v, float) else v)
962                                   for k, v in r.items()})
963         agg_count = sum(1 for c in m_cols if c.endswith("__mean") or c.endswith("__std"))
964         print(f"Wrote {m_path}: {len(movie_records)} movies × {agg_count} aggregated features",
965               flush=True)
966
967     # Summary
968     import re
969     groups = {
970         "OpenFace (of_)": "^of_",
971         "Quantum (q_)": "^q_",
972         "Empatica (emp_)": "^emp_",
973         "EEG (eeg_)": "^eeg_",
974         "Samsung (sw_)": "^sw_",
975         "Self-Report (sr_)": "^sr_",
976         "Participant": "^(participant_|wearing_|minutes_|viewing_)",
977         "Missing masks": "^has_",
978     }
979     print(f"\n{'='*60}", flush=True)
980     print(f" V6 Feature Extraction Complete", flush=True)
981     print(f"{'='*60}", flush=True)
982     print(f" Participant-level: {len(all_records)} × {len(feature_cols)}", flush=True)
983     print(f" Movie-level: {len(movie_records)} × {agg_count}", flush=True)
984     print(f"\n Feature breakdown:", flush=True)
985     for label, pat in groups.items():
986         n = sum(1 for c in feature_cols if re.match(pat, c))
987         print(f"    {label:25s} {n:3d}", flush=True)
988
989 if __name__ == "__main__":
990     main()

```

Simulate 40 movies (ML, n = 50 corpus)

ml_dataset/scripts/simulate_40_movies_ML.py

```
1 #!/usr/bin/env python3
2 """
3 SIMULATION OF 40 SYNTHETIC MOVIES FOR ML TRAINING
4
5 Approach: Two-level hierarchical class-conditional sampling using
6 gaussian copulas. The copula approach preserves BOTH:
7 - Each feature's marginal distribution (by mapping through the empirical CDF)
8 - The correlation structure between features (by sampling correlated normals)
9
10 Outputs 3 file
11 - scene_movie_metadata_v6_synthetic.csv (40 rows)
12 - participant_features_v6_synthetic.csv (40 × 43 = 1720 rows)
13 - movie_features_v6_synthetic.csv (40 rows)
14
15 All synthetic rows include is_synthetic=1.
16 """
17 from __future__ import annotations
18 import csv
19 import math
20 import random
21 import statistics
22 from pathlib import Path
23 from collections import Counter, defaultdict
24 import warnings
25
26 import numpy as np
27
28 warnings.filterwarnings("ignore")
29
30 # -----
31 PROJECT = Path(__file__).resolve().parent.parent.parent
32 DATA_DIR = PROJECT / "ml_dataset" / "data" / "model_ready" / "movie_success_v6"
33
34 RANDOM_SEED = 42
35 N_SYNTHETIC = 40
36 N_PARTICIPANTS = 43
37 SYNTHETIC_PARTICIPANT_ID_START = 1000
38 TIER_THRESHOLD = 7.5
39 CPI_BASE = 322.0
40 CPI = {1978: 65.2, 1979: 72.6, 1983: 99.6, 1988: 118.3, 1993: 144.5,
41        1996: 156.9, 1998: 163.0, 1999: 166.6, 2006: 201.6, 2012: 229.6,
42        2025: 322.0}
43
44 random.seed(RANDOM_SEED)
45 np.random.seed(RANDOM_SEED)
46
47 def phi(z):
48     """Standard normal CDF (vectorised over numpy array)."""
49     z = np.asarray(z, dtype=float)
50     return 0.5 * (1.0 + np.vectorize(math.erf)(z / math.sqrt(2.0)))
51
52
53 def phi_inv(p):
54     """
55     Inverse standard normal CDF using Beasley-Springer-Moro approximation.
56     Vectorised over numpy array. Accuracy ~1e-9 in mid-range.
57     """
58     p = np.clip(np.asarray(p, dtype=float), 1e-12, 1 - 1e-12)
59
60     a = [-39.6968302866538, 220.946098424521, -275.928510446969,
61          138.357751867269, -30.6647980661472, 2.50662827745924]
62     b = [-54.4760987982241, 161.585836858041, -155.698979859887,
63          66.8013118877197, -13.2806815528857]
64     c = [-0.00778489400243029, -0.322396458041136, -2.40075827716184,
65          -2.54973253934373, 4.37466414146497, 2.93816398269878]
66     d = [0.00778469570904146, 0.32246712907004, 2.445134137143, 3.75440866190742]
67
68     p_low = 0.02425
69     p_high = 1 - p_low
70     out = np.empty_like(p)
71
72     # Mid region
73     mid = (p >= p_low) & (p <= p_high)
```

```

74     if np.any(mid):
75         q = p[mid] - 0.5
76         r = q * q
77         num = (((a[0]*r + a[1])*r + a[2])*r + a[3])*r + a[4]*r + a[5]
78         den = (((b[0]*r + b[1])*r + b[2])*r + b[3])*r + b[4]*r + 1
79         out[mid] = num * q / den
80
81     # Lower tail
82     low = p < p_low
83     if np.any(low):
84         q = np.sqrt(-2 * np.log(p[low]))
85         num = (((c[0]*q + c[1])*q + c[2])*q + c[3])*q + c[4]*q + c[5]
86         den = (((d[0]*q + d[1])*q + d[2])*q + d[3])*q + 1
87         out[low] = num / den
88
89     # Upper tail (mirror of lower)
90     high = p > p_high
91     if np.any(high):
92         q = np.sqrt(-2 * np.log(1 - p[high]))
93         num = (((c[0]*q + c[1])*q + c[2])*q + c[3])*q + c[4]*q + c[5]
94         den = (((d[0]*q + d[1])*q + d[2])*q + d[3])*q + 1
95         out[high] = -num / den
96
97     return out
98
99
100 # -----
101 # COPULA UTILITIES
102 # -----
103
104 def empirical_ranks(values):
105     """
106     Return rank-based percentiles in (0, 1), avoiding boundaries.
107     Uses (rank + 0.5) / n convention; tied values get the same rank.
108     """
109     n = len(values)
110     if n == 0:
111         return np.array([])
112     # argsort-of-argsort gives 0-indexed rank
113     ranks = np.argsort(np.argsort(values)).astype(float)
114     return (ranks + 0.5) / n
115
116
117 def inverse_empirical(percentiles, sorted_values):
118     """Linear-interpolating inverse ECDF: percentile [0,1] → value in original units."""
119     n = len(sorted_values)
120     if n == 0:
121         return np.zeros_like(percentiles)
122     if n == 1:
123         return np.full_like(percentiles, sorted_values[0])
124     idx = percentiles * (n - 1)
125     idx = np.clip(idx, 0, n - 1)
126     lo = np.floor(idx).astype(int)
127     hi = np.minimum(lo + 1, n - 1)
128     frac = idx - lo
129     return sorted_values[lo] * (1.0 - frac) + sorted_values[hi] * frac
130
131
132 def fit_copula(X, regularize=0.05):
133     """
134     Fit a Gaussian copula on real data X (n × p).
135     Returns model with: cholesky factor, sorted real values per feature,
136     and a flag indicating constant features.
137     """
138     n, p = X.shape
139
140     sorted_per_feat = []
141     constant_value = [] # for features with zero variance
142     is_constant = np.zeros(p, dtype=bool)
143
144     Z = np.zeros((n, p))
145     for j in range(p):
146         col = X[:, j]
147         valid_mask = ~np.isnan(col)
148         valid = col[valid_mask]
149
150         if len(valid) == 0:
151             sorted_per_feat.append(np.array([0.0]))

```



```

152         constant_value.append(0.0)
153         is_constant[j] = True
154         continue
155
156     sorted_col = np.sort(valid)
157     sorted_per_feat.append(sorted_col)
158
159     # Constant feature
160     if sorted_col[0] == sorted_col[-1]:
161         is_constant[j] = True
162         constant_value.append(sorted_col[0])
163         Z[:, j] = 0.0
164         continue
165     constant_value.append(0.0)
166
167     # Compute z-scores via empirical CDF  $\rightarrow \Phi^{-1}$ 
168     valid_ranks = empirical_ranks(valid)
169     Z_valid = phi_inv(valid_ranks)
170
171     # Place Z_valid back into the full column; impute NaN as 0 (median in z-space)
172     z_col = np.zeros(n)
173     z_col[valid_mask] = Z_valid
174     z_col[~valid_mask] = 0.0
175     Z[:, j] = z_col
176
177 # Correlation matrix of Z
178 # Skip constant columns when computing correlation
179 var = Z.var(axis=0)
180 nonconst = var > 1e-10
181 Z_nc = Z[:, nonconst]
182 C_nc = np.corrcoef(Z_nc, rowvar=False) if Z_nc.shape[1] > 1 else np.eye(1)
183 C_nc = np.nan_to_num(C_nc, nan=0.0)
184
185 # Reconstruct full correlation matrix: identity for constant columns
186 C = np.eye(p)
187 nc_indices = np.where(nonconst)[0]
188 for i_idx, i in enumerate(nc_indices):
189     for j_idx, j in enumerate(nc_indices):
190         C[i, j] = C_nc[i_idx, j_idx]
191
192 # Regularise: shrink toward identity for numerical stability
193 C = (1 - regularize) * C + regularize * np.eye(p)
194
195 # Cholesky decomposition (with extra regularisation if needed)
196 for extra in [0.0, 0.05, 0.2, 0.5]:
197     try:
198         C_try = (1 - extra) * C + extra * np.eye(p)
199         L = np.linalg.cholesky(C_try)
200         break
201     except np.linalg.LinAlgError:
202         continue
203 else:
204     raise RuntimeError("Cholesky failed even with heavy regularisation.")
205
206 return {
207     "L": L, "p": p, "n": n,
208     "sorted_per_feat": sorted_per_feat,
209     "is_constant": is_constant,
210     "constant_value": constant_value,
211 }
212
213
214 def sample_copula(model, n_samples):
215     """Draw n_samples x p synthetic feature vectors from the fitted copula."""
216     L = model["L"]
217     p = model["p"]
218     sorted_per_feat = model["sorted_per_feat"]
219     is_constant = model["is_constant"]
220     constant_value = model["constant_value"]
221
222     # Step 1: correlated standard normals
223     noise = np.random.standard_normal((n_samples, p))
224     Z_new = noise @ L.T
225
226     # Step 2:  $\rightarrow$  uniform via  $\Phi$ 
227     U_new = phi(Z_new)
228
229     # Step 3:  $\rightarrow$  original space via inverse ECDF

```

```

230 X_new = np.empty((n_samples, p))
231 for j in range(p):
232     if is_constant[j]:
233         X_new[:, j] = constant_value[j]
234     else:
235         X_new[:, j] = inverse_empirical(U_new[:, j], sorted_per_feat[j])
236
237 return X_new
238
239
240
241 def load_csv(path):
242     with open(path) as f:
243         reader = csv.DictReader(f)
244         return reader.fieldnames, list(reader)
245
246 def to_float(s):
247     if s is None or s == "" or s == "None":
248         return float("nan")
249     try:
250         return float(s)
251     except ValueError:
252         return float("nan")
253
254 #LOAD REAL DATA
255
256 meta_cols, meta_rows = load_csv(DATA_DIR / "scene_movie_metadata_v6.csv")
257 print(f" Metadata: {len(meta_rows)} movies x {len(meta_cols)} cols")
258
259 ppt_cols, ppt_rows = load_csv(DATA_DIR / "participant_features_v6.csv")
260 print(f" Participants: {len(ppt_rows)} rows x {len(ppt_cols)} cols")
261
262 movie_imdb = {r["movie_id"]: float(r["imdb_rating"]) for r in meta_rows
263               if r.get("imdb_rating")}
264 tier_of_movie = {mid: ("high" if rating >= TIER_THRESHOLD else "lower")
265                  for mid, rating in movie_imdb.items()}
266
267 movies_in_tier = defaultdict(list)
268 for mid, t in tier_of_movie.items():
269     movies_in_tier[t].append(mid)
270
271 print("\nTier assignment:")
272 for t in ["high", "lower"]:
273     mids = movies_in_tier[t]
274     ratings = [movie_imdb[m] for m in mids]
275     print(f" {t:6s}: {len(mids)} movies, IMDb {min(ratings):.1f}--{max(ratings):.1f}")
276
277
278
279 #PREPARE PARTICIPANT-LEVEL FEATURES PER TIER
280
281 ID_COLS = {"participant_id", "condition", "movie_id"}
282 HAS_COLS = {c for c in ppt_cols if c.startswith("has_")}
283 SR_INTEGER_COLS = {c for c in ppt_cols
284                    if c.startswith("sr_emo_") and not c.startswith("sr_emo_bc_")}
285 SR_INTEGER_COLS.update({"sr_sam_valence", "sr_sam_arousal", "sr_sam_motivation"})
286
287 feat_cols = [c for c in ppt_cols if c not in ID_COLS]
288 n_feat = len(feat_cols)
289
290 tier_data = defaultdict(list)
291 for r in ppt_rows:
292     mid = r["movie_id"]
293     if mid not in tier_of_movie:
294         continue
295     vec = np.array([to_float(r[c]) for c in feat_cols])
296     tier_data[tier_of_movie[mid]].append(vec)
297
298 print(f"\nFeature matrix sizes:")
299 for t, vecs in tier_data.items():
300     arr = np.vstack(vecs)
301     print(f" Tier '{t}': {arr.shape[0]} x {arr.shape[1]}, NaN rate {np.isnan(arr).mean():.1%}")
302
303
304
305 #FIT GAUSSIAN COPULAS
306
307

```

```

308 print("\nFitting Gaussian copulas (preserves marginals + correlations)...")
309 tier_models = {}
310 for t, vecs in tier_data.items():
311     X = np.vstack(vecs)
312     model = fit_copula(X, regularize=0.05)
313     tier_models[t] = model
314     n_const = int(model["is_constant"].sum())
315     print(f"Tier '{t}': fitted on {model['n']} rows x {model['p']} features "
316           f"({n_const} constant features)")
317
318
319
320 #METADATA SAMPLING DICTIONARIES
321
322
323 tier_metadata = defaultdict(lambda: defaultdict(list))
324 for r in meta_rows:
325     mid = r["movie_id"]
326     if mid not in tier_of_movie:
327         continue
328     for col in meta_cols:
329         tier_metadata[tier_of_movie[mid]][col].append(r[col])
330
331 CATEGORICAL_SCENE = ["cut_count", "brightness", "motion_intensity", "audio_loudness",
332                      "silence_ratio", "music_presence", "dialogue_density",
333                      "face_screen_time_ratio", "lead_screen_time_ratio",
334                      "budget_categorical"]
335
336
337
338 #POST-PROCESS SAMPLED FEATURES
339
340
341 def post_process_row(vec, feat_cols):
342     """Force has_* masks → 1, round integer self-report features."""
343     out = vec.copy().astype(float)
344     for i, c in enumerate(feat_cols):
345         if c in HAS_COLS:
346             out[i] = 1.0
347         elif c.startswith("sr_emo_") and not c.startswith("sr_emo_bc_"):
348             out[i] = round(np.clip(out[i], 1, 5))
349         elif c in {"sr_sam_valence", "sr_sam_arousal", "sr_sam_motivation"}:
350             out[i] = round(np.clip(out[i], 1, 9))
351         elif c.startswith("sr_emo_bc_"):
352             out[i] = round(np.clip(out[i], -4, 4))
353     return out
354
355
356
357 #GENERATE SYNTHETIC MOVIES
358
359
360 def sample_categorical(values):
361     return random.choice(values)
362
363 n_per_tier = N_SYNTHETIC // 2
364 print(f"\nGenerating {N_SYNTHETIC} synthetic movies ({n_per_tier} per tier)...")
365
366 synthetic_metadata = []
367 synthetic_participants = []
368 synthetic_movie_features = []
369
370 syn_idx = 0
371 for tier in ["high", "lower"]:
372     model = tier_models[tier]
373     real_imdb = [float(v) for v in tier_metadata[tier]["imdb_rating"]]
374     real_revenue = [int(v) for v in tier_metadata[tier]["worldwide_gross_revenue_usd_orignal_release"]]
375     real_wom_log = [float(v) for v in tier_metadata[tier]["wom_multiplier_log"]]
376     real_year = [int(v) for v in tier_metadata[tier]["release_year"]]
377     real_duration = [int(v) for v in tier_metadata[tier]["clip_duration_s"]]
378
379     imdb_min, imdb_max = min(real_imdb), max(real_imdb)
380     rev_log_mean = statistics.mean([math.log10(v) for v in real_revenue])
381     rev_log_std = statistics.stdev([math.log10(v) for v in real_revenue])
382     wom_log_mean = statistics.mean(real_wom_log)
383     wom_log_std = statistics.stdev(real_wom_log)
384
385     for i in range(n_per_tier):

```

```

386     syn_idx += 1
387     syn_movie_id = f"SYN_M{syn_idx:03d}"
388     syn_scene_id = f"SYN_S{syn_idx:03d}"
389
390     # Sample 43 synthetic participants from the copula
391     ppts = sample_copula(model, N_PARTICIPANTS)
392     ppts = np.array([post_process_row(p, feat_cols) for p in ppts])
393
394     syn_emotion = sample_categorical(tier_metadata[tier]["targeted_emotion"])
395     syn_year = sample_categorical(real_year)
396     syn_duration = sample_categorical(real_duration)
397     syn_genre = sample_categorical(tier_metadata[tier]["genre_primary"])
398     syn_genre2 = sample_categorical(tier_metadata[tier]["genre_secondary"])
399     syn_country = sample_categorical(tier_metadata[tier]["country_of_origin"])
400     syn_budget_cat = sample_categorical(tier_metadata[tier]["budget_categorical"])
401     syn_imdb = round(random.uniform(imdb_min, imdb_max), 1)
402
403     syn_rev = max(100_000, int(10 ** np.random.normal(rev_log_mean, rev_log_std)))
404     syn_wom_log = np.random.normal(wom_log_mean, wom_log_std)
405     syn_wom = 10 ** syn_wom_log
406     syn_opening = max(10_000, int(syn_rev / syn_wom))
407
408     cpi_year = CPI.get(syn_year, 200.0)
409     inflate = CPI_BASE / cpi_year
410     syn_rev_2025 = round(syn_rev * inflate, 2)
411     syn_opn_2025 = round(syn_opening * inflate, 2)
412
413     syn_categoricals = {col: sample_categorical(tier_metadata[tier][col])
414                        for col in CATEGORICAL_SCENE}
415
416     meta_row = {
417         "scene_id": syn_scene_id,
418         "movie_id": syn_movie_id,
419         "movie_title": f"Synthetic_Movie_{syn_idx:03d}",
420         "imdb_link": "",
421         "targeted_emotion": syn_emotion,
422         "clip_duration_s": syn_duration,
423         "release_year": syn_year,
424         "genre_primary": syn_genre,
425         "genre_secondary": syn_genre2,
426         "country_of_origin": syn_country,
427         "imdb_rating": syn_imdb,
428         "worldwide_gross_revenue_usd_orignal_release": syn_rev,
429         "opening_weekend_usd": syn_opening,
430         "revenue_usd_2025": syn_rev_2025,
431         "opening_weekend_usd_2025": syn_opn_2025,
432         "wom_multiplier": round(syn_wom, 4),
433         "wom_multiplier_log": round(syn_wom_log, 4),
434         "scene_notes": f"Synthetic – class-conditional copula sample from {tier} IMDb tier",
435         "data_collection_notes": (
436             f"Synthetic movie {syn_idx} of 40 (Gaussian copula sampling, "
437             f"tier='{tier}', seed=42). Marginals match real distributions; "
438             f"correlation structure preserved."
439         ),
440         "is_synthetic": 1,
441     }
442     meta_row.update(syn_categoricals)
443     synthetic_metadata.append(meta_row)
444
445     # Participant-level rows
446     for p_i in range(N_PARTICIPANTS):
447         syn_pid = SYNTHETIC_PARTICIPANT_ID_START + (syn_idx - 1) * N_PARTICIPANTS + p_i
448         row = {"participant_id": syn_pid, "condition": syn_emotion, "movie_id": syn_movie_id}
449         for j, c in enumerate(feat_cols):
450             v = float(ppts[p_i, j])
451             if abs(v) < 1e-6:
452                 row[c] = 0
453             else:
454                 row[c] = round(v, 6)
455         row["is_synthetic"] = 1
456         synthetic_participants.append(row)
457
458     # Movie-level aggregation
459     m_row = {
460         "movie_id": syn_movie_id,
461         "condition": syn_emotion,
462         "n_participants": N_PARTICIPANTS,
463         "imdb_rating": syn_imdb,

```

```

464         "wom_multiplier": round(syn_wom, 4),
465         "wom_multiplier_log": round(syn_wom_log, 4),
466         "is_synthetic": 1,
467     }
468     for j, c in enumerate(feats_cols):
469         col_vals = ppts[:, j]
470         m_row[f"{c}__mean"] = round(float(col_vals.mean()), 6)
471         m_row[f"{c}__std"] = round(float(col_vals.std(ddof=1)), 6)
472     synthetic_movie_features.append(m_row)
473
474
475
476 #WRITE OUTPUTS
477
478
479 syn_meta_cols = list(meta_cols) + ["is_synthetic"]
480 out_meta = DATA_DIR / "scene_movie_metadata_v6_synthetic.csv"
481 with open(out_meta, "w", newline="") as f:
482     writer = csv.DictWriter(f, fieldnames=syn_meta_cols, extrasaction="ignore")
483     writer.writeheader()
484     writer.writerows(synthetic_metadata)
485 print(f"\nWrote {out_meta.name}: {len(synthetic_metadata)} rows x {len(syn_meta_cols)} cols")
486
487 syn_ppt_cols = list(ppt_cols) + ["is_synthetic"]
488 out_ppt = DATA_DIR / "participant_features_v6_synthetic.csv"
489 with open(out_ppt, "w", newline="") as f:
490     writer = csv.DictWriter(f, fieldnames=syn_ppt_cols, extrasaction="ignore")
491     writer.writeheader()
492     writer.writerows(synthetic_participants)
493 print(f"Wrote {out_ppt.name}: {len(synthetic_participants)} rows x {len(syn_ppt_cols)} cols")
494
495 mov_cols, _ = load_csv(DATA_DIR / "movie_features_v6.csv")
496 syn_mov_cols = list(mov_cols)
497 if "is_synthetic" not in syn_mov_cols:
498     syn_mov_cols.append("is_synthetic")
499 for c in ("imdb_rating", "wom_multiplier", "wom_multiplier_log"):
500     if c not in syn_mov_cols:
501         syn_mov_cols.append(c)
502
503 out_mov = DATA_DIR / "movie_features_v6_synthetic.csv"
504 with open(out_mov, "w", newline="") as f:
505     writer = csv.DictWriter(f, fieldnames=syn_mov_cols, extrasaction="ignore")
506     writer.writeheader()
507     writer.writerows(synthetic_movie_features)
508 print(f"Wrote {out_mov.name}: {len(synthetic_movie_features)} rows x {len(syn_mov_cols)} cols")
509
510 # 8. VALIDATION SUMMARY
511
512 print("\n" + "=" * 60)
513 print(" VALIDATION - synthetic vs real distributions")
514 print("=" * 60)
515
516 real_imdb_all = list(movie_imdb.values())
517 syn_imdb_all = [m["imdb_rating"] for m in synthetic_metadata]
518 print(f"\nIMDb rating:")
519 print(f" Real (n={len(real_imdb_all)}): mean={statistics.mean(real_imdb_all):.2f}, "
520       f"std={statistics.stdev(real_imdb_all):.2f}, "
521       f"range={min(real_imdb_all):.1f}-{max(real_imdb_all):.1f}")
522 print(f" Synthetic (n={len(syn_imdb_all)}): mean={statistics.mean(syn_imdb_all):.2f}, "
523       f"std={statistics.stdev(syn_imdb_all):.2f}, "
524       f"range={min(syn_imdb_all):.1f}-{max(syn_imdb_all):.1f}")
525
526 real_wom_log = [float(r["wom_multiplier_log"]) for r in meta_rows]
527 syn_wom_log = [m["wom_multiplier_log"] for m in synthetic_metadata]
528 print(f"\nWOM multiplier (log10):")
529 print(f" Real (n={len(real_wom_log)}): mean={statistics.mean(real_wom_log):.2f}, "
530       f"std={statistics.stdev(real_wom_log):.2f}, "
531       f"range={min(real_wom_log):.2f}-{max(real_wom_log):.2f}")
532 print(f" Synthetic (n={len(syn_wom_log)}): mean={statistics.mean(syn_wom_log):.2f}, "
533       f"std={statistics.stdev(syn_wom_log):.2f}, "
534       f"range={min(syn_wom_log):.2f}-{max(syn_wom_log):.2f}")
535
536 print(f"\nDONE. Combined corpus (real + synthetic) = 50 movies.")

```

Simulate 10,000 movies (DL, n = 10,010 corpus)

ml_dataset/scripts/simulate_10k_movies_DL.py

```
1 #!/usr/bin/env python3
2 """
3 SIMULATION OF 10,000 SYNTHETIC MOVIES FOR DEEP LEARNING TRAINING
4
5 Simulate 10,000 synthetic movies using the same Gaussian copula
6 methodology scaled up for deep-learning training.
7
8 Two-level hierarchical class-conditional sampling using
9 gaussian copulas.
10 The copula preserves BOTH:
11 - Each feature's marginal distribution (by mapping through the empirical CDF)
12 - The correlation structure between features (by sampling correlated normals)
13
14 Outputs 3 files in ml_dataset/data/model_ready/movie_success_v7/:
15 - scene_movie_metadata_v7_synthetic.csv (10,000 rows)
16 - participant_features_v7_synthetic.csv (10,000 × 43 = 430,000 rows)
17 - movie_features_v7_synthetic.csv (10,000 rows)
18
19 All synthetic rows include is_synthetic=1.
20 """
21 from __future__ import annotations
22 import csv
23 import math
24 import random
25 import statistics
26 from pathlib import Path
27 from collections import Counter, defaultdict
28 import warnings
29
30 import numpy as np
31
32 warnings.filterwarnings("ignore")
33
34 PROJECT = Path(__file__).resolve().parent.parent.parent
35 INPUT_DIR = PROJECT / "ml_dataset" / "data" / "model_ready" / "movie_success_v6"
36 DATA_DIR = PROJECT / "ml_dataset" / "data" / "model_ready" / "movie_success_v7"
37 DATA_DIR.mkdir(parents=True, exist_ok=True)
38
39 RANDOM_SEED = 42
40 N_SYNTHETIC = 10000
41 N_PARTICIPANTS = 43
42 SYNTHETIC_PARTICIPANT_ID_START = 100000
43 TIER_THRESHOLD = 7.5
44 CPI_BASE = 322.0
45 CPI = {1978: 65.2, 1979: 72.6, 1983: 99.6, 1988: 118.3, 1993: 144.5,
46        1996: 156.9, 1998: 163.0, 1999: 166.6, 2006: 201.6, 2012: 229.6,
47        2025: 322.0}
48
49 random.seed(RANDOM_SEED)
50 np.random.seed(RANDOM_SEED)
51
52
53 def phi(z):
54     """Standard normal CDF (vectorised over numpy array)."""
55     z = np.asarray(z, dtype=float)
56     return 0.5 * (1.0 + np.vectorize(math.erf)(z / math.sqrt(2.0)))
57
58
59 def phi_inv(p):
60     """
61     Inverse standard normal CDF using Beasley-Springer-Moro approximation.
62     Vectorised over numpy array. Accuracy ~1e-9 in mid-range.
63     """
64     p = np.clip(np.asarray(p, dtype=float), 1e-12, 1 - 1e-12)
65
66     a = [-39.6968302866538, 220.946098424521, -275.928510446969,
67          138.357751867269, -30.6647980661472, 2.50662827745924]
68     b = [-54.4760987982241, 161.585836858041, -155.698979859887,
69          66.8013118877197, -13.2806815528857]
70     c = [-0.00778489400243029, -0.322396458041136, -2.40075827716184,
71          -2.54973253934373, 4.37466414146497, 2.93816398269878]
72     d = [0.00778469570904146, 0.32246712907004, 2.445134137143, 3.75440866190742]
```

```

74 p_low = 0.02425
75 p_high = 1 - p_low
76 out = np.empty_like(p)
77
78 # Mid region
79 mid = (p >= p_low) & (p <= p_high)
80 if np.any(mid):
81     q = p[mid] - 0.5
82     r = q * q
83     num = (((a[0]*r + a[1])*r + a[2])*r + a[3])*r + a[4])*r + a[5]
84     den = (((b[0]*r + b[1])*r + b[2])*r + b[3])*r + b[4])*r + 1
85     out[mid] = num * q / den
86
87 # Lower tail
88 low = p < p_low
89 if np.any(low):
90     q = np.sqrt(-2 * np.log(p[low]))
91     num = (((c[0]*q + c[1])*q + c[2])*q + c[3])*q + c[4])*q + c[5]
92     den = (((d[0]*q + d[1])*q + d[2])*q + d[3])*q + 1
93     out[low] = num / den
94
95 # Upper tail (mirror of lower)
96 high = p > p_high
97 if np.any(high):
98     q = np.sqrt(-2 * np.log(1 - p[high]))
99     num = (((c[0]*q + c[1])*q + c[2])*q + c[3])*q + c[4])*q + c[5]
100    den = (((d[0]*q + d[1])*q + d[2])*q + d[3])*q + 1
101    out[high] = -num / den
102
103    return out
104
105 # COPULA UTILITIES
106
107 def empirical_ranks(values):
108     """
109     Return rank-based percentiles in (0, 1), avoiding boundaries.
110     Uses (rank + 0.5) / n convention; tied values get the same rank.
111     """
112     n = len(values)
113     if n == 0:
114         return np.array([])
115     # argsort-of-argsort gives 0-indexed rank
116     ranks = np.argsort(np.argsort(values)).astype(float)
117     return (ranks + 0.5) / n
118
119
120 def inverse_empirical(percentiles, sorted_values):
121     """Linear-interpolating inverse ECDF: percentile [0,1] → value in original units."""
122     n = len(sorted_values)
123     if n == 0:
124         return np.zeros_like(percentiles)
125     if n == 1:
126         return np.full_like(percentiles, sorted_values[0])
127     idx = percentiles * (n - 1)
128     idx = np.clip(idx, 0, n - 1)
129     lo = np.floor(idx).astype(int)
130     hi = np.minimum(lo + 1, n - 1)
131     frac = idx - lo
132     return sorted_values[lo] * (1.0 - frac) + sorted_values[hi] * frac
133
134
135 def fit_copula(X, regularize=0.05):
136     """
137     Fit a Gaussian copula on real data X (n × p).
138     Returns model with: cholesky factor, sorted real values per feature,
139     and a flag indicating constant features.
140     """
141     n, p = X.shape
142
143     sorted_per_feat = []
144     constant_value = [] # for features with zero variance
145     is_constant = np.zeros(p, dtype=bool)
146
147     Z = np.zeros((n, p))
148     for j in range(p):
149         col = X[:, j]
150         valid_mask = ~np.isnan(col)
151         valid = col[valid_mask]

```



```

152
153     if len(valid) == 0:
154         sorted_per_feat.append(np.array([0.0]))
155         constant_value.append(0.0)
156         is_constant[j] = True
157         continue
158
159     sorted_col = np.sort(valid)
160     sorted_per_feat.append(sorted_col)
161
162     # Constant feature
163     if sorted_col[0] == sorted_col[-1]:
164         is_constant[j] = True
165         constant_value.append(sorted_col[0])
166         Z[:, j] = 0.0
167         continue
168     constant_value.append(0.0)
169
170     # Compute z-scores via empirical CDF  $\rightarrow \Phi^{-1}$ 
171     valid_ranks = empirical_ranks(valid)
172     Z_valid = phi_inv(valid_ranks)
173
174     # Place Z_valid back into the full column; impute NaN as 0 (median in z-space)
175     z_col = np.zeros(n)
176     z_col[valid_mask] = Z_valid
177     z_col[~valid_mask] = 0.0
178     Z[:, j] = z_col
179
180     # Correlation matrix of Z
181     # Skip constant columns when computing correlation
182     var = Z.var(axis=0)
183     nonconst = var > 1e-10
184     Z_nc = Z[:, nonconst]
185     C_nc = np.corrcoef(Z_nc, rowvar=False) if Z_nc.shape[1] > 1 else np.eye(1)
186     C_nc = np.nan_to_num(C_nc, nan=0.0)
187
188     # Reconstruct full correlation matrix: identity for constant columns
189     C = np.eye(p)
190     nc_indices = np.where(nonconst)[0]
191     for i_idx, i in enumerate(nc_indices):
192         for j_idx, j in enumerate(nc_indices):
193             C[i, j] = C_nc[i_idx, j_idx]
194
195     # Regularise: shrink toward identity for numerical stability
196     C = (1 - regularize) * C + regularize * np.eye(p)
197
198     # Cholesky decomposition (with extra regularisation if needed)
199     for extra in [0.0, 0.05, 0.2, 0.5]:
200         try:
201             C_try = (1 - extra) * C + extra * np.eye(p)
202             L = np.linalg.cholesky(C_try)
203             break
204         except np.linalg.LinAlgError:
205             continue
206     else:
207         raise RuntimeError("Cholesky failed even with heavy regularisation.")
208
209     return {
210         "L": L, "p": p, "n": n,
211         "sorted_per_feat": sorted_per_feat,
212         "is_constant": is_constant,
213         "constant_value": constant_value,
214     }
215
216
217 def sample_copula(model, n_samples):
218     """Draw n_samples x p synthetic feature vectors from the fitted copula."""
219     L = model["L"]
220     p = model["p"]
221     sorted_per_feat = model["sorted_per_feat"]
222     is_constant = model["is_constant"]
223     constant_value = model["constant_value"]
224
225     # Step 1: correlated standard normals
226     noise = np.random.standard_normal((n_samples, p))
227     Z_new = noise @ L.T
228
229     # Step 2:  $\rightarrow$  uniform via  $\Phi$ 

```

```

230     U_new = phi(Z_new)
231
232     # Step 3: → original space via inverse ECDF
233     X_new = np.empty((n_samples, p))
234     for j in range(p):
235         if is_constant[j]:
236             X_new[:, j] = constant_value[j]
237         else:
238             X_new[:, j] = inverse_empirical(U_new[:, j], sorted_per_feat[j])
239
240     return X_new
241
242
243 def load_csv(path):
244     with open(path) as f:
245         reader = csv.DictReader(f)
246         return reader.fieldnames, list(reader)
247
248 def to_float(s):
249     if s is None or s == "" or s == "None":
250         return float("nan")
251     try:
252         return float(s)
253     except ValueError:
254         return float("nan")
255
256 # 1. LOAD REAL DATA
257 print("Loading real data...")
258
259 meta_cols, meta_rows = load_csv(INPUT_DIR / "scene_movie_metadata_v6.csv")
260 print(f" Metadata: {len(meta_rows)} movies × {len(meta_cols)} cols")
261
262 ppt_cols, ppt_rows = load_csv(INPUT_DIR / "participant_features_v6.csv")
263 print(f" Participants: {len(ppt_rows)} rows × {len(ppt_cols)} cols")
264
265 movie_imdb = {r["movie_id"]: float(r["imdb_rating"]) for r in meta_rows
266               if r.get("imdb_rating")}
267 tier_of_movie = {mid: ("high" if rating >= TIER_THRESHOLD else "lower")
268                 for mid, rating in movie_imdb.items()}
269
270 movies_in_tier = defaultdict(list)
271 for mid, t in tier_of_movie.items():
272     movies_in_tier[t].append(mid)
273
274 print("\nTier assignment:")
275 for t in ["high", "lower"]:
276     mids = movies_in_tier[t]
277     ratings = [movie_imdb[m] for m in mids]
278     print(f" {t:6s}: {len(mids)} movies, IMDb {min(ratings):.1f}–{max(ratings):.1f}")
279
280 # 2. PREPARE PARTICIPANT-LEVEL FEATURES PER TIER
281
282 ID_COLS = {"participant_id", "condition", "movie_id"}
283 HAS_COLS = {c for c in ppt_cols if c.startswith("has_")}
284 SR_INTEGER_COLS = {c for c in ppt_cols
285                   if c.startswith("sr_emo_") and not c.startswith("sr_emo_bc_")}
286 SR_INTEGER_COLS.update({"sr_sam_valence", "sr_sam_arousal", "sr_sam_motivation"})
287
288 feat_cols = [c for c in ppt_cols if c not in ID_COLS]
289 n_feat = len(feat_cols)
290
291 tier_data = defaultdict(list)
292 for r in ppt_rows:
293     mid = r["movie_id"]
294     if mid not in tier_of_movie:
295         continue
296     vec = np.array([to_float(r[c]) for c in feat_cols])
297     tier_data[tier_of_movie[mid]].append(vec)
298
299 print(f"\nFeature matrix sizes:")
300 for t, vecs in tier_data.items():
301     arr = np.vstack(vecs)
302     print(f" Tier '{t}': {arr.shape[0]} × {arr.shape[1]}, NaN rate {np.isnan(arr).mean():.1%}")
303
304 # 3. FIT GAUSSIAN COPULAS
305
306 print("\nFitting Gaussian copulas (preserves marginals + correlations)...")
307 tier_models = {}

```

```

308 for t, vecs in tier_data.items():
309     X = np.vstack(vecs)
310     model = fit_copula(X, regularize=0.05)
311     tier_models[t] = model
312     n_const = int(model["is_constant"].sum())
313     print(f" Tier '{t}': fitted on {model['n']} rows x {model['p']} features "
314           f"({n_const} constant features)")
315
316
317 # 4. METADATA SAMPLING DICTIONARIES
318
319 tier_metadata = defaultdict(lambda: defaultdict(list))
320 for r in meta_rows:
321     mid = r["movie_id"]
322     if mid not in tier_of_movie:
323         continue
324     for col in meta_cols:
325         tier_metadata[tier_of_movie[mid]][col].append(r[col])
326
327 CATEGORICAL_SCENE = ["cut_count", "brightness", "motion_intensity", "audio_loudness",
328                      "silence_ratio", "music_presence", "dialogue_density",
329                      "face_screen_time_ratio", "lead_screen_time_ratio",
330                      "budget_categorical"]
331
332
333 # 5. POST-PROCESS SAMPLED FEATURES
334
335 def post_process_row(vec, feat_cols):
336     """Force has_* masks → 1, round integer self-report features."""
337     out = vec.copy().astype(float)
338     for i, c in enumerate(feat_cols):
339         if c in HAS_COLS:
340             out[i] = 1.0
341         elif c.startswith("sr_emo_") and not c.startswith("sr_emo_bc_"):
342             out[i] = round(np.clip(out[i], 1, 5))
343         elif c in {"sr_sam_valence", "sr_sam_arousal", "sr_sam_motivation"}:
344             out[i] = round(np.clip(out[i], 1, 9))
345         elif c.startswith("sr_emo_bc_"):
346             out[i] = round(np.clip(out[i], -4, 4))
347     return out
348
349 # 6. GENERATE SYNTHETIC MOVIES
350
351 def sample_categorical(values):
352     return random.choice(values)
353
354 n_per_tier = N_SYNTHETIC // 2
355 print(f"\nGenerating {N_SYNTHETIC} synthetic movies ({n_per_tier} per tier)...")
356
357 synthetic_metadata = []
358 synthetic_participants = []
359 synthetic_movie_features = []
360
361 syn_idx = 0
362 for tier in ["high", "lower"]:
363     model = tier_models[tier]
364     real_imdb = [float(v) for v in tier_metadata[tier]["imdb_rating"]]
365     real_revenue = [int(v) for v in tier_metadata[tier]["worldwide_gross_revenue_usd_ordinal_release"]]
366     real_wom_log = [float(v) for v in tier_metadata[tier]["wom_multiplier_log"]]
367     real_year = [int(v) for v in tier_metadata[tier]["release_year"]]
368     real_duration = [int(v) for v in tier_metadata[tier]["clip_duration_s"]]
369
370     imdb_min, imdb_max = min(real_imdb), max(real_imdb)
371     rev_log_mean = statistics.mean([math.log10(v) for v in real_revenue])
372     rev_log_std = statistics.stdev([math.log10(v) for v in real_revenue])
373     wom_log_mean = statistics.mean(real_wom_log)
374     wom_log_std = statistics.stdev(real_wom_log)
375
376     for i in range(n_per_tier):
377         syn_idx += 1
378         syn_movie_id = f"SYN_M{syn_idx:03d}"
379         syn_scene_id = f"SYN_S{syn_idx:03d}"
380
381         # Sample 43 synthetic participants from the copula
382         ppts = sample_copula(model, N_PARTICIPANTS)
383         ppts = np.array([post_process_row(p, feat_cols) for p in ppts])
384
385         syn_emotion = sample_categorical(tier_metadata[tier]["targeted_emotion"])

```

```

386 syn_year = sample_categorical(real_year)
387 syn_duration = sample_categorical(real_duration)
388 syn_genre = sample_categorical(tier_metadata[tier]["genre_primary"])
389 syn_genre2 = sample_categorical(tier_metadata[tier]["genre_secondary"])
390 syn_country = sample_categorical(tier_metadata[tier]["country_of_origin"])
391 syn_budget_cat = sample_categorical(tier_metadata[tier]["budget_categorical"])
392 syn_imdb = round(random.uniform(imdb_min, imdb_max), 1)
393
394 syn_rev = max(100_000, int(10 ** np.random.normal(rev_log_mean, rev_log_std)))
395 syn_wom_log = np.random.normal(wom_log_mean, wom_log_std)
396 syn_wom = 10 ** syn_wom_log
397 syn_opening = max(10_000, int(syn_rev / syn_wom))
398
399 cpi_year = CPI.get(syn_year, 200.0)
400 inflate = CPI_BASE / cpi_year
401 syn_rev_2025 = round(syn_rev * inflate, 2)
402 syn_opn_2025 = round(syn_opening * inflate, 2)
403
404 syn_categoricals = {col: sample_categorical(tier_metadata[tier][col])
405                     for col in CATEGORICAL_SCENE}
406
407 meta_row = {
408     "scene_id": syn_scene_id,
409     "movie_id": syn_movie_id,
410     "movie_title": f"Synthetic_Movie_{syn_idx:03d}",
411     "imdb_link": "",
412     "targeted_emotion": syn_emotion,
413     "clip_duration_s": syn_duration,
414     "release_year": syn_year,
415     "genre_primary": syn_genre,
416     "genre_secondary": syn_genre2,
417     "country_of_origin": syn_country,
418     "imdb_rating": syn_imdb,
419     "worldwide_gross_revenue_usd_orignal_release": syn_rev,
420     "opening_weekend_usd": syn_opening,
421     "revenue_usd_2025": syn_rev_2025,
422     "opening_weekend_usd_2025": syn_opn_2025,
423     "wom_multiplier": round(syn_wom, 4),
424     "wom_multiplier_log": round(syn_wom_log, 4),
425     "scene_notes": f"Synthetic – class-conditional copula sample from {tier} IMDb tier",
426     "data_collection_notes": (
427         f"Synthetic movie {syn_idx} of 40 (Gaussian copula sampling, "
428         f"tier='{tier}', seed=42). Marginals match real distributions; "
429         f"correlation structure preserved."
430     ),
431     "is_synthetic": 1,
432 }
433 meta_row.update(syn_categoricals)
434 synthetic_metadata.append(meta_row)
435
436 # Participant-level rows
437 for p_i in range(N_PARTICIPANTS):
438     syn_pid = SYNTHETIC_PARTICIPANT_ID_START + (syn_idx - 1) * N_PARTICIPANTS + p_i
439     row = {"participant_id": syn_pid, "condition": syn_emotion, "movie_id": syn_movie_id}
440     for j, c in enumerate(feat_cols):
441         v = float(ppts[p_i, j])
442         if abs(v) < 1e-6:
443             row[c] = 0
444         else:
445             row[c] = round(v, 6)
446     row["is_synthetic"] = 1
447     synthetic_participants.append(row)
448
449 # Movie-level aggregation
450 m_row = {
451     "movie_id": syn_movie_id,
452     "condition": syn_emotion,
453     "n_participants": N_PARTICIPANTS,
454     "imdb_rating": syn_imdb,
455     "wom_multiplier": round(syn_wom, 4),
456     "wom_multiplier_log": round(syn_wom_log, 4),
457     "is_synthetic": 1,
458 }
459 for j, c in enumerate(feat_cols):
460     col_vals = ppts[:, j]
461     m_row[f"{c}__mean"] = round(float(col_vals.mean()), 6)
462     m_row[f"{c}__std"] = round(float(col_vals.std(ddof=1)), 6)
463 synthetic_movie_features.append(m_row)

```

```

464
465 # 7. WRITE OUTPUTS
466
467 syn_meta_cols = list(meta_cols) + ["is_synthetic"]
468 out_meta = DATA_DIR / "scene_movie_metadata_v7_synthetic.csv"
469 with open(out_meta, "w", newline="") as f:
470     writer = csv.DictWriter(f, fieldnames=syn_meta_cols, extrasaction="ignore")
471     writer.writeheader()
472     writer.writerows(synthetic_metadata)
473 print(f"\nWrote {out_meta.name}: {len(synthetic_metadata)} rows x {len(syn_meta_cols)} cols")
474
475 syn_ppt_cols = list(ppt_cols) + ["is_synthetic"]
476 out_ppt = DATA_DIR / "participant_features_v7_synthetic.csv"
477 with open(out_ppt, "w", newline="") as f:
478     writer = csv.DictWriter(f, fieldnames=syn_ppt_cols, extrasaction="ignore")
479     writer.writeheader()
480     writer.writerows(synthetic_participants)
481 print(f"Wrote {out_ppt.name}: {len(synthetic_participants)} rows x {len(syn_ppt_cols)} cols")
482
483 mov_cols, _ = load_csv(INPUT_DIR / "movie_features_v6.csv")
484 syn_mov_cols = list(mov_cols)
485 if "is_synthetic" not in syn_mov_cols:
486     syn_mov_cols.append("is_synthetic")
487 for c in ("imdb_rating", "wom_multiplier", "wom_multiplier_log"):
488     if c not in syn_mov_cols:
489         syn_mov_cols.append(c)
490
491 out_mov = DATA_DIR / "movie_features_v7_synthetic.csv"
492 with open(out_mov, "w", newline="") as f:
493     writer = csv.DictWriter(f, fieldnames=syn_mov_cols, extrasaction="ignore")
494     writer.writeheader()
495     writer.writerows(synthetic_movie_features)
496 print(f"Wrote {out_mov.name}: {len(synthetic_movie_features)} rows x {len(syn_mov_cols)} cols")
497
498 # 8. VALIDATION SUMMARY
499
500 print("\n" + "=" * 60)
501 print(" VALIDATION - synthetic vs real distributions")
502 print("=" * 60)
503
504 real_imdb_all = list(movie_imdb.values())
505 syn_imdb_all = [m["imdb_rating"] for m in synthetic_metadata]
506 print(f"\nIMDb rating:")
507 print(f" Real (n={len(real_imdb_all)}): mean={statistics.mean(real_imdb_all):.2f}, "
508       f"std={statistics.stdev(real_imdb_all):.2f}, "
509       f"range={min(real_imdb_all):.1f}--{max(real_imdb_all):.1f}")
510 print(f" Synthetic (n={len(syn_imdb_all)}): mean={statistics.mean(syn_imdb_all):.2f}, "
511       f"std={statistics.stdev(syn_imdb_all):.2f}, "
512       f"range={min(syn_imdb_all):.1f}--{max(syn_imdb_all):.1f}")
513
514 real_wom_log = [float(r["wom_multiplier_log"]) for r in meta_rows]
515 syn_wom_log = [m["wom_multiplier_log"] for m in synthetic_metadata]
516 print(f"\nWOM multiplier (log10):")
517 print(f" Real (n={len(real_wom_log)}): mean={statistics.mean(real_wom_log):.2f}, "
518       f"std={statistics.stdev(real_wom_log):.2f}, "
519       f"range={min(real_wom_log):.2f}--{max(real_wom_log):.2f}")
520 print(f" Synthetic (n={len(syn_wom_log)}): mean={statistics.mean(syn_wom_log):.2f}, "
521       f"std={statistics.stdev(syn_wom_log):.2f}, "
522       f"range={min(syn_wom_log):.2f}--{max(syn_wom_log):.2f}")
523
524 print(f"\nDONE. Combined corpus (real + synthetic) = 50 movies.")

```